Математическая логика и теория алгоритмов

Альтернативные модели вычислений

Машина Поста, модель $\mathcal{P}'$

Эквивалентность моделей и тезис Чёрча—Тьюринга

- Эквивалентность моделей: *любое вычисление, которое можно выполнить с помощью одной модели вычислений, можно выполнить и средствами другой*.
- Для любой появляющейся новой модели вычислений достаточно доказать её эквивалентность только одной из тех моделей, для которых эквивалентность остальным уже была установлена. Очень часто для этой цели используется машина Тьюринга, как наиболее убедительная модель, описывающая работу человека-вычислителя в чрезвычайно простых операциях.
- Модель квантовых вычислений: её эквивалентность машинам Тьюринга уже доказана.

Тезис Чёрча—Тьюринга:

- ***любая вычислимая функция может быть вычислена с помощью соответствующей машины Тьюринга или любой другой эквивалентной ей модели***.
- Это утверждение не может быть доказано, поскольку в нём участвует неопределяемое понятие «***вычислимой функции***», под которым понимается нечто, что может быть принципиально вычислено, какими бы методами мы при этом ни пользовались.



Эквивалентность моделей и тезис Чёрча—Тьюринга

- Тезис Чёрча—Тьюринга принимается на веру: он отражает наши представления о современной картине мира, его устройстве с точки зрения процесса вычислений. Он также позволяет нам утверждать, что если что-то не может быть вычислено в одной из разобранных ранее моделей вычислений, то это что-то *не может быть вычислено в принципе*.
- Таким образом, мы получаем способ решения проблем - проблем о доказательстве несуществования алгоритмов решения определённых задач. Достаточно доказать, что не существует, к примеру, машины Тьюринга, которая бы их решала. Поскольку это понятие формально определено, то д*оказательство несуществования оказывается возможным*.
- Непосредственное использование вычислительных моделей крайне затруднительно, они для этого слишком упрощены. Вместо этого на практике используются полные по Тьюрингу языки программирования (таковыми являются абсолютное большинство известных языков программирования). В некотором смысле они тоже являются моделями вычислений, но снабжены большим количеством *синтаксического сахара*, который облегчает их использование, но затрудняет формальные рассуждения относительно семантики программ.

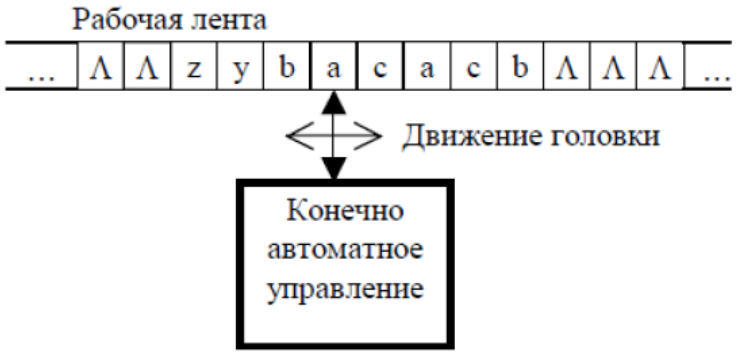
Эквивалентность моделей и тезис Чёрча—Тьюринга

- Известно, что любой язык программирования с возможностью ветвления и условным циклом (например, while) или неограниченной рекурсией, полон по Тьюрингу (то есть представляет собой эквивалентную машине Тьюринга модель вычислений).
- Интересно, что иногда используют и неполные по Тьюрингу языки программирования. К ним относятся языки, программы на которых гарантированно завершаются (например, Coq или Agda). В некоторых языках требование завершаемости может быть явно предъявлено к некоторой функции, в этом случае компилятор выполняет соответствующую проверку (неразрешимая задача). Примером таких языков является язык программирования Idris. Требование завершаемости необходимо в случаях, когда критически важна корректность работы программного обеспечения.

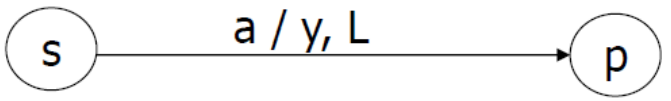


Машины Тьюринга

Алан Тьюринг в 1936г. предложил формальную модель вычислителя, которая является результатом простого добавления потенциально бесконечной памяти к системе управления с конечной памятью



Программа МТ – конечное множество пятерок
<s, a, p, y, D>, или
(s, a) ⟶ (p, y, D)



МТ = (S, Σ, s_0, δ, Г)
Г = {L, R, H};
δ: S × Σ ⟶ S × Σ × Г

Программа:
1. (s, a) ⟶ (p, y, L);
2. (q, b) ⟶ (s, z, R);
3. …

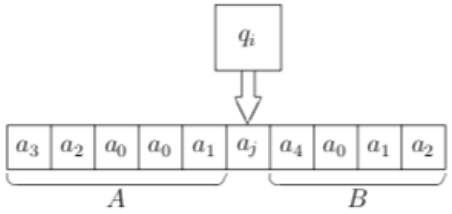
S – состояния
Σ - вход. алфавит
s_0 – начальное состояние
δ – функция переходов
Г – движение головки

Команды:
(состояние, вход) → (состояние, выход, D∈Г)

Недетерминированные машины Тьюринга

- Обычные машины Тьюринга называют также ***детерминированными*** машинами Тьюринга, подчеркивая тем самым, что в программе любой такой машины для каждого состояния q_i ($i > 0$) и любого внешнего символа a_j существует ровно одна команда вида $\mathbf{q_i a_j \rightarrow \ldots}$
- Процесс вычисления на такой машине представляется как *последовательность конфигураций*, каждая из которых **однозначно** определялась предыдущей.
- По аналогии с недетерминированными конечными автоматами определяется обобщенное понятие недетерминированной машины Тьюринга, которая отличается от детерминированного варианта тем, что в ее программе для любого q_i ($i > 0$) и любого a_j может быть несколько (или нисколько) команд вида $q_i a_j \rightarrow \ldots$
- Это означает, что, находясь в состоянии q_i и обозревая на ленте символ a_j, машина может выполнить любую из этих команд.
- Если же в состоянии q_i, обозревая символ a_j, машина не находит в программе команду вида $q_i a_j \rightarrow \ldots$, то дальнейшие шаги в данной ветви вычислений невозможны.
- Таким образом, шаг работы машины уже *не определен однозначно*, а процесс вычисления можно представлять как **дерево конфигураций**, в котором из одной конфигурации может получиться уже несколько других.
- Каждая ветвь дерева конфигураций либо является бесконечной, либо заканчивается в заключительном состоянии, либо обрывается на некотором не заключительном состоянии. В последнем случае говорят, что произошла *неправильная остановка* машины.

Недетерминированная машина Тьюринга



Упорядоченная пятерка $T = \langle A, Q, P, q_1, q_0 \rangle$, где

- $A = \{a_0, a_1, \ldots, a_n\}$ – конечный внешний алфавит;
- $Q = \{q_0, q_1, \ldots, q_m\}$ — конечный алфавит внутренних состояний;
- $P = \{T(i, j) \mid 1 \le i \le m, 0 \le j \le n\}$ — программа, состоящая из команд $T(i, j)$, каждая из которых есть слово вида: $q_i a_j \to q_k a_l$, или $q_i a_j \to q_k a_l R$, или $q_i a_j \to q_k a_l L$, где $0 \le k \le m$, $0 \le l \le n$;
- q_1 — начальное состояние;
- q_0 — конечное состояние.

- Программой называется любое подмножество

$$P \subseteq (Q \setminus \{q_0\}) \times A \times Q \times A \times \{\Lambda, R, L\}.$$

- Элементы программы P называются *командами*, и каждую команду $\langle q_i, a_j, q_k, a_l, S \rangle \in P$ мы записываем в виде слова $q_i a_j \to q_k a_l S$.
- В программе P может быть несколько команд вида $q_i a_j \to \ldots$
- Допускается также случай, когда в P команды, начинающиеся на $q_i a_j \to \ldots$, отсутствуют.

Распознавание языка

- Если недетерминированная машина T распознает язык L, то все ее вычисления, начатые на входном слове $w \in A^*$, заканчиваются (это может произойти по двум причинам: либо в вычислении происходит ***правильная остановка***, либо происходит ***неправильная остановка***).
- При этом если хотя бы одно вычисление заканчивается в состоянии q_0 и выдает 1, то считается, что слово w распозналось. Даже если в данной ситуации среди остальных ветвей вычислений найдутся такие, которые выдают 0 в состоянии q_0 или заканчиваются ввиду неправильной остановки, машина по определению распознает слово w.



Пример

Рассмотрим язык $L = \{a^n \mid n$ — составное число$\}$ над алфавитом $A_0=\{a\}$.

- Обычная детерминированная машина Тьюринга, распознающая данный язык, действует следующим образом:
  - если на вход подано слово a_n, $n > 0$, то машина последовательно перебирает числа $k = 2,3,...,[\sqrt{n}]$, и для каждого k проверяет, делится ли n на число k. Если хотя бы одно k делит n, то число n — составное, иначе — простое (или меньше двойки).

Недетерминированная машина Тьюринга устроена проще:

- Благодаря недетерминированности циклический перебор чисел $k = 2, 3, \ldots, [\sqrt{n}]$ можно разбить на несколько ветвей параллельных вычислений — для каждого значения k своя ветвь. Если хотя бы одна ветвь окажется успешной, то число — составное. Если все ветви неуспешные, то число — несоставное.
- Внешний алфавит машины, кроме символов *a*, *0*, *1*, будет содержать вспомогательный символ *b*.

Пример

- Допустим, на вход машины подано слово **0aaaaaaaaaaaa0** (т. е. n = 12 в данном частном случае).
- Машина пытается недетерминированно «угадать» нетривиальный делитель k числа n, для этого она, используя символ b, отмечает в входном слове начальный префикс длины k следующим образом: ***0bb0aaaaaaaaa0*** (т. е. k = 3 в данном частном случае). После этого машина пытается «исчерпать» выбранным префиксом bb0 все буквы a, передвигая его слева направо
  - 0bb0aaaaaaaaa0 $\vdash*$ 0000bb0aaaaaa0 $\vdash*$ 0000000bb0aaa0 $\vdash*$ . . .
- Если входное слово удается покрыть кратным количеством слов вида bb0, то «до- гадка» машины оказалась правильной. Иначе, т. е. если n не делится нацело на k, данную ветвь вычислений можно считать неуспешной.

Программа для примера

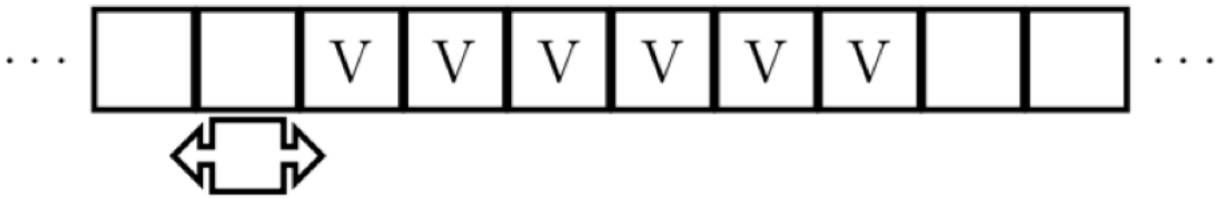
$$
\begin{array}{ll}
q_1 0 \to q_2 0 R & q_8 b \to q_8 b L \\
q_2 a \to q_3 b R & q_8 0 \to q_9 0 L \\
q_3 a \to q_3 b R & q_9 b \to q_4 b \\
q_3 a \to q_4 0 L & q_9 0 \to q_{10} 0 R \\
q_4 b \to q_4 b L & q_{10} 0 \to q_{11} 0 R \\
q_4 0 \to q_5 0 R & q_{11} b \to q_{11} b R \\
q_5 b \to q_6 0 R & q_{11} a \to q_{12} 0 R \\
q_6 b \to q_6 b R & q_{12} 0 \to q_0 1 \\
q_6 0 \to q_7 0 R & q_{12} a \to q_{13} a L \\
q_7 b \to q_7 b R & q_{13} 0 \to q_4 0 L \\
q_7 a \to q_8 b L &
\end{array}
$$

- Автомат построить самостоятельно!

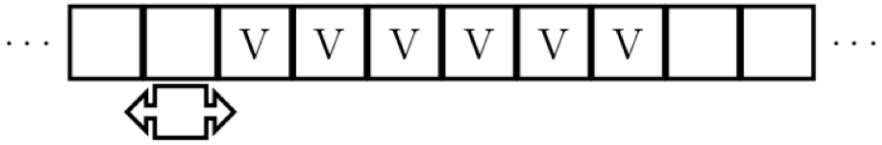


Машина Поста

- Машина Поста —модель вычислений, похожая на машину Тьюринга, но использующая более примитивные операции и имеющая более простую структуру.
- Предложена Эмилем Постом в 1936 году, создана независимо от машины Тьюринга, но сообщение о машине Поста опубликовано на несколько месяцев позднее. Отличается от машины Тьюринга большей простотой, притом обе машины алгоритмически «эквивалентны» и обе разработаны для формализации понятия алгоритма и решения задач об алгоритмической разрешимости.
- Машина Поста состоит из ***бесконечной ленты***, каждая ячейка которой может быть *помеченной*, либо *нет* (понятие ленточного алфавита при этом не вводится).
- Одна из ячеек ленты считается в каждый момент времени *текущей* (её обозревает каретка считывающего устройства).
- Помимо команды останова машина Поста *останавливается* при выполнении недопустимой операции (установка метки в уже помеченную ячейку, снятие метки с непомеченной, переход на несуществующую команду).
- При работе машины Поста возможно *зацикливание*.







Машина Поста

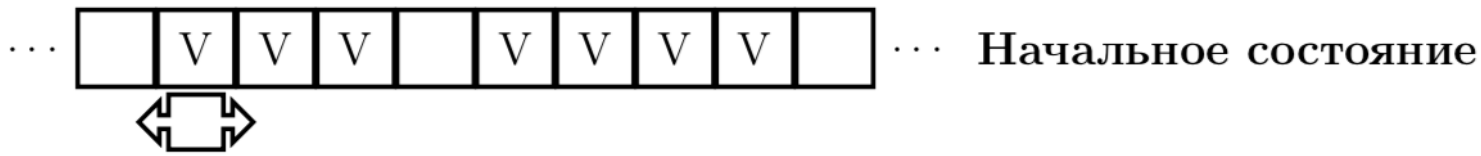
Программой для машины Поста является конечная пронумерованная последовательность команд:

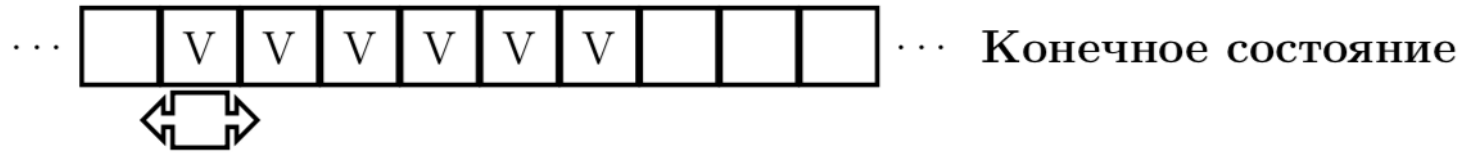
- **V** $_j$ — установить метку и перейти на команду с номером j;
- **X** $_j$ — стереть метку и перейти на команду с номером j;
- **L** $_j$ — сдвинуть каретку влево и перейти на команду с номером j;
- **R** $_j$ — сдвинуть каретку вправо и перейти на команду с номером j;
- **?** j_1, j_2 — если текущая ячейка помечена, то перейти на команду с номером j_1, иначе перейти на команду с номером j_2;
- **!** — остановить работу машины.

Пример

Пример сложения двух натуральных чисел.

- Для представления числа n будем использовать последовательность из n + 1 подряд идущих помеченных ячеек (одинокая помеченная ячейка соответствует числу 0); числа будут отделяться друг от друга одной непомеченной ячейкой.
- Начальная позиция каретки — первая помеченная ячейка первого числа, на ней же будем завершать работу машины.

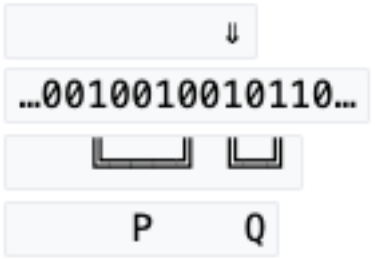




Пример

- Для решения задачи нужно будет:
  - а) найти и установить метку в ячейку, разделяющую числа (команды 1-3);
  - б) найти и стереть две метки в конце второго числа (команды 4-9);
  - в) вернуться на первую ячейку результата (команды 10-13).

```
1   R 2          8   L 9
2   ? 1,3        9   X 10
3   V 4          10  L 11
4   R 5          11  ? 10,12
5   ? 4,6        12  R 13
6   L 7          13  !
7   X 8
```



Пример

Программа вычитания натуральных (целых неотрицательных) **P** и **Q** чисел состоит из последовательного изменения крайних левых «1» у представления *Q* и правых «1» у представления *P*. В начале программы каретка установлена на крайнюю левую «1».

- **1.** ← — шаг влево
- **2. ? 1; 3** — если в ячейке пусто, перейти к 1-шагу, если нет — к 3.
- **3. X** — удалить метку.
- **4.** → — шаг вправо.
- **5. ? 4; 6** — если в ячейке пусто, перейти к 4-шагу, если нет — к 6.
- **6. X** — удалить метку.
- **7.** → — шаг вправо.
- **8. ? 9; 1** — если в ячейке пусто, перейти к 9 шагу, если нет — к 1.
- **9. !** — конец.

В 5-й строке возможно зацикливание, если Q > P.

Машины Шёнфилда

Машина Шёнфилда — это механическое вычислительное устройство, хотя окончательная его формализация является совершенно математической. Ограничимся только «механическим» описанием.

Машина Шёнфилда однозначно задается:

- 1) Бесконечным множеством регистров, пронумерованных натуральными числами 0,1,2,...
  - Каждый регистр — это ячейка памяти, способная содержать любое натуральное число.
  - Содержимое регистров может меняться по ходу вычислений. Отметим, что каждая конкретная машина Шёнфилда использует в своих вычислениях только конечное число регистров. Основное назначение регистровой памяти — это хранение входных, промежуточных и выходных данных.
- 2) Счетчиком команд, являющимся особой ячейкой памяти, которая в каждый данный момент содержит некоторое натуральное число. Счетчик команд указывает на номер команды в программе, которая исполняется в данный момент. В начальный момент в счетчик команд заносится 0.
- 3) Программой, содержащейся в отдельной выделенной памяти машины. Программа — это конечный список команд, последовательно пронумерованных натуральными числами от 0 до некоторого $n \in \omega$.
  В ходе вычислений программа не меняется (фиксирована).

Машины Шёнфилда

Шаг машины состоит в исполнении той команды из программы, номер которой указан в данный момент в счетчике команд.

- Если команды с таким номером в программе **НЕТ**, то машина ***останавливается***.

Существует только два типа команд:

- **INC i**
  машина увеличивает содержимое i-го регистра и счетчик команд на 1, после чего переходит к следующему шагу.
- **DEC i, n**
  Если к началу исполнения данной команды содержимое i-го регистра больше 0, то машина уменьшает его на 1 и заносит *n* в счетчик команд. Если же в данный момент содержимое i-го регистра равно 0, то машина увеличивает счетчик команд на 1.



Не исключены ситуации, в которых машина работает бесконечно, т. е. не останавливается!

Расширения машины Шёнфилда

- Чтобы вычислительные возможности исходных машин не изменились, добавим только те команды, которые допускают реализацию в виде программы, использующей только команды типа ***INC i*** и ***DEC i, n***. Любую дополнительную команду, обладающую таким свойством, будем называть **макросом**.
- Любой макрос получается по следующему принципу: если P — обычная программа, то ей сопоставляется оператор P^*, который называется макросом и который может исполняться как отдельная команда. Программы, содержащие макросы, будем называть **макропрограммами**.

Исполнение макросов

- Исполнение макроса P^*, находящегося в макропрограмме в строке под номером m, происходит следующим образом:
  - 1) Если в счетчике команд находится m, то в машину (в качестве подпрограммы) вызывается программа P.
  - 2) P исполняется с теми данными в регистрах, которые там содержались в данный момент.
  - 3) Если работа программы P закончилась, то в счетчик команд заносится m + 1 и исполнение макроса на этом заканчивается. Если же P работает бесконечно, то макропрограмма также не останавливается.

- Любой макрос может использовать конечный набор параметров, каждый из которых является номером некоторого регистра (эти номера обычно присутствуют в имени макроса). Макросы не могут использовать в качестве своих параметров номера строк программы.
- Макросы — это лишь удобный инструмент для сокращения текстов программ.
- Макропрограммы P и Q называются *эквивалентными*, если запуски машин Шёнфилда с этими макропрограммами на одних и тех же начальных содержимых регистров либо приведут к остановке обеих машин с одинаковыми содержимыми всех регистров, либо обе машины никогда не остановятся.

- Теорема (об элиминации макросов). ***Любая макропрограмма эквивалентна программе, не содержащей макросы.***

Примеры

- Исполнение последовательной пары команд

  0 : INC 0 и 1 : DEC 0, n

  просто заносит n в счетчик команд, при этом содержимые регистров не меняются. Однако нельзя сопоставить этой короткой программе макрос, использующий параметр n, поскольку n здесь является номером строки.
- Программа с простейшим циклом

  0 : DEC i, 0

  обнуляет содержимое i-го регистра. Обозначим макрос, соответствующий этой программе, через ***ZERO i***.

Пример

- Пусть i, j, k — фиксированные натуральные числа такие, что i ≠ k и j ≠ k. Обозначим через **[i] → [j], (k)** макрос, который копирует содержимое i-го регистра в j-й регистр, используя k-й регистр как вспомогательный. Содержимое i-го регистра не изменяется. Программа, реализующая данный макрос при i ≠ j, имеет следующий вид:

```
0 :  DEC j, 0
1 :  DEC k, 1
2 :  INC 0
3 :  DEC 0, 6
4 :  INC j
5 :  INC k
6 :  DEC i, 4
7 :  INC 0
8 :  DEC 0, 10
9 :  INC i
10 : DEC k, 9
```

- При i = j можно взять любую программу, которая ничего не меняет, например:

```
0 :  INC 0
1 :  DEC 0, 2
```

Еще один пример

- Обозначим через **ADD [1] TO [0], (3)** макрос, при исполнении которого содержимое 1-го регистра прибавляется к содержимому 0-го регистра. При этом содержимое 1-го регистра сохраняется, а 3-й регистр используется как вспомогательный.

```
0 :  DEC 3, 0
1 :  INC 0
2 :  DEC 0, 5
3 :  INC 0
4 :  INC 3
5 :  DEC 1, 3
6 :  INC 0
7 :  DEC 0, 9
8 :  INC 1
9 :  DEC 3, 8
```

- С помощью введенного макроса макропрограмму, перемножающую содержимые 1-го и 2-го регистров и записывающую полученное произведение в 0-й регистр:

```
0 :  DEC 0, 0
1 :  INC 0
2 :  DEC 0, 4
3 :  ADD [1] TO [0], (3)
4 :  DEC 2, 3
```

Продолжение «еще одного примера»

- Если применить к данной макропрограмме процедуру преобразования, то получим эквивалентную ей программу без макросов (теорема!):

```
0 :  DEC 0, 0
1 :  INC 0
2 :  DEC 0, 13
3 :  DEC 3, 3
4 :  INC 0
5 :  DEC 0, 8
6 :  INC 0
7 :  INC 3
8 :  DEC 1, 6
9 :  INC 0
10 : DEC 0, 12
11 : INC 1
12 : DEC 3, 11
13 : DEC 2, 3
```

Вычислимость на машине Шёнфилда

k-местная частичная функция $f : M \subseteq \omega^k \to \omega$ называется ***вычислимой на машине Шёнфилда***, если существует машина Шёнфилда с программой P такая, что для любых $x_1, \ldots, x_k \in \omega$ выполняется:

- а) если $f(x_1,\ldots,x_k)$ определено, то после запуска машины с начальными значениями $x_1, \ldots, x_k$ в регистрах $1, \ldots, k$ соответственно и с нулями в остальных регистрах машина после конечного числа шагов останавливается, и после остановки в 0-ом регистре находится $f(x_1,\ldots,x_k)$;
- б) если $f(x_1,\ldots,x_k)$ не определено, то после запуска машины с начальными значениямих $x_1,\ldots,x_k$ в регистрах $1,\ldots,k$ соответственно и с нулями в остальных регистрах машина никогда не останавливается и работает бесконечно.

- Если f – нульместная функция, то входные данные $x_1, \ldots, x_k$ отсутствуют, а результат вычислений по программе P не зависит от начальных содержимых регистров.

Вычислимость на машине Шёнфилда

- Пусть f – k-местная частичная функция, вычислимая на машине Шёнфилда с программой P. Обозначим через
  $$\mathbf{f([i_1],\ldots,[i_k]) \rightarrow [j],\ s}$$
  макрос P*, который вычисляет значение функции f от содержимых регистров с номерами $i_1,\ldots,i_k$, записывает его в j-й регистр и при этом не изменяет содержимое всех регистров до s-го включительно, кроме j-го регистра.
- Если упомянутое значение функции не определено, то исполнение данного макроса приводит к бесконечной работе программы.
- Пусть m – произвольное число, превосходящее числа $i_1, \ldots, i_k$, j, s и номера всех регистров, входящих в программу P.
- Результат исполнения макроса $P*$ не зависит от содержимых регистров, номера которых больше, чем $m-1$.
- Параметр s можно заранее выбрать достаточно большим, чтобы не влиять на работу внешней макропрограммы.

Макропрограмма

$[0] \to [m+1],(m)$ $\dots$ $[m-1] \to [m+m],(m)$	сохраняем содержимое регистров с 0-го до $(m-1)$-го в регистрах с номерами от $m+1$ до $2m$
$[m+1+i_1] \to [1],(m)$ $\dots$ $[m+1+i_k] \to [k],(m)$ ZERO 0 ZERO $k+1$ $\dots$ ZERO $m-1$	подготавливаем входные данные для исполнения макроса P^*
P^*	вычисляем необходимое значение функции f
$[m+2] \to [1],(m)$ $\dots$ $[m+m] \to [m-1],(m)$	восстанавливаем содержимые регистров с номерами от 1 до $m-1$
$[0] \to [j],(m)$	заносим результат вычислений в j-й регистр
$[m+1] \to [0],(m)$	если $j \neq 0$, то восстанавливаем содержимое 0-го регистра (если $j=0$, то эта срока не включается в программу!)

Машины Шёнфилда и предикаты

Любая функция, вычислимая по Шёнфилду, является *частично рекурсивной*.

Код команды машины Шёнфилда:

- код(INC i) = < 0, i >
- код(DEC i, j) = < 1, i, j >

Код программы, состоящей из команд $c_0,...,c_{k-1}$, как следующее натуральное число

- < код(c_0), . . . , код(c_{k-1}) >

Тогда определим трехместный предикат:

- **Stop(e, x, n)** $\Leftarrow\Rightarrow$ Выполняются следующие три условия:
  - (а) e — код некоторой программы P,
  - (б) x =< x_1, . . . , x_k > — код некоторой последовательности натуральных чисел,
  - (в) программа P, начав работу с содержимыми регистров 0, x_1,..., x_k, 0, 0,..., останавливается после n-го шага.

Вернемся к тезису Чёрча

Рассмотрели четыре различные формализации понятия вычислимой функции:

- частично рекурсивные функции,
- машины Шёнфилда,
- машины Тьюринга (Поста),
- нормальные алгорифмы Маркова.

- Данные подходы отталкиваются от разных принципов и имеют не похожие друг на друга реализации (за исключением, быть может, под- ходов Тьюринга и Шёнфилда).
- Существуют ***другие***, менее распространенные формализации понятия вычислимости. Однако все известные формальные подходы порождают один и тот же класс функций.

- **ТЕОРЕМА.** Для произвольной частичной функции $f : dom(f) \subseteq \omega^n \to \omega$ следующие условия эквивалентны:
  - 1) f является частично рекурсивной функцией;
  - 2) f вычислима на машине Шёнфилда;
  - 3) f вычислима по Тьюрингу;
  - 4) f вычислима по Маркову.

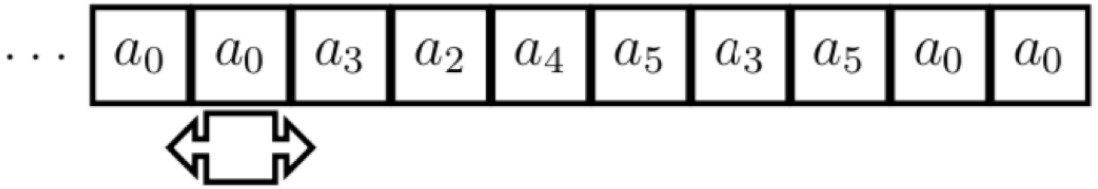
Алонзо Чёрч
1903-1995



Тезис Чёрча

- **Тезис Чёрча.** Класс интуитивно вычислимых функций совпадает с классом всех частично рекурсивных функций.
- Тезис Чёрча – высказывание, которое говорит, что предложенная формализация понятия вычислимой функции адекватно отражает наши интуитивные представления о вычислимости. Исторически именно этот тезис был первым точным определением частично вычислимой функции.
- Тезис Чёрча не является формальным математическим утверждением. Однако при доказательстве теорем он часто используется в неявном виде.
- Чтобы доказать существование частично рекурсивной функции с определенными свойствами, в математических рассуждениях может быть показана ее интуитивная вычислимость, а строгое доказательство частичной рекурсивности при этом опускается. Такой прием позволяет сократить объем доказательства, но автор любого такого доказательства должен быть готов представить формальный вариант доказательства, не использующий тезис Чёрча.

Модель $\mathcal{P}''$



- С точки зрения языков программирования машины Поста и Тьюринга, а также регистровые машины ***чересчур ориентированы на инструкцию goto***.
- В период становления структурного программирования один из его пионеров итальянский специалист **Коррадо Бём** решил сформулировать эквивалентную модель вычислений, ориентированную на структурное программирование. Результатом этой работы явилась модель $\mathcal{P}''$ (1964 год). Получился первый полный по Тьюрингу язык программирования без оператора GOTO.
- В этой модели используется *полубесконечная влево* лента, в каждой позиции которой находится один символ ленточного алфавита $\{a_0, a_1, \ldots, a_n\}$, где n>=1.
- Символ a_0 считается пробельным, так что в алфавите имеется как минимум один непробельный символ.

Модель $\mathcal{P}''$

Программа в модели $\mathcal{P}''$ — это слово в алфавите $\{R, \lambda, (,)\}$, строящееся по следующим правилам:

- 1) R и λ — слова;
- 2) если p и q — слова, то pq — слово;
- 3) если q — слово, то (q) — слово;
- 4) других слов нет.

Следующие программы являются **синтаксически корректными**:

- $RR\lambda(RR\lambda)$;
- $((\lambda\lambda)(RR))$;
- $R\lambda R\lambda(R)(\lambda)$.

Семантика, то есть порядок обработки этих символов при исполнении программы, следующая:

- R — сдвинуть каретку вправо на одну позицию (если это возможно);
- λ — заменить текущий символ a_i на $a_{(i+1)\bmod(n+1)}$ и сдвинуть каретку влево на одну позицию;
- (q) — пока текущий символ не является пробельным (a_0), повторять q.

Модель $\mathcal{P}''$

- Символ λ выполняет *циклический инкремент* текущего символа, то есть символ a_n заменяется на a_0, а все остальные символы заменяются на следующий символ ленточного алфавита.
- *Круглые скобки* являются аналогом условного цикла while, так что здесь действительно удалось отказаться от использования конструкции, так или иначе похожей на goto.
- Символы программы обрабатываются *слева направо*, выполнение программы **завершается** после обработки последнего символа.
- Можно показать, что введённых команд **достаточно** для реализации произвольной вычислимой функции, однако пользоваться ими не очень удобно.

Следующий набор команд также достаточен, но гораздо более удобен (часть из них можно считать *синтаксическим сахаром*):

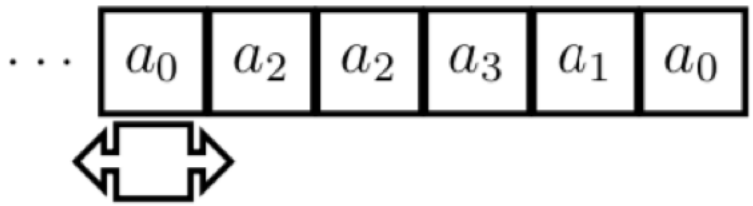
- $r \equiv \lambda R$ — инкремент символа без сдвига каретки;
- $r' \equiv r^n \equiv \underbrace{rr \ldots r}_{n \text{ раз}}$ — декремент символа;
- $L \equiv r'\lambda$ — сдвиг влево без изменения символа;
- R — сдвиг вправо.

Пример

Пользуясь этим набором команд, реализуем вычитание единицы из числа, заданного в n-ичной системе счисления:

- символ a_1 соответствует цифре 0, символ a_2 — цифре 1 и т.д.;
- младший разряд числа записывается справа;
- число ограничено пробельными символами слева и справа.

Лента модели $\mathcal{P}''$ с записанным на ней числом 42 в *троичной системе* счисления (размер ленточного алфавита n = 3):

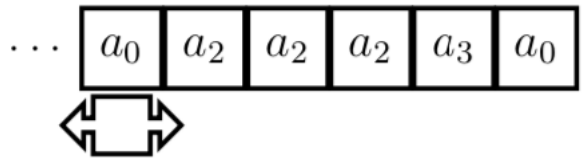


Пример

- Будем считать, что перед началом работы каретка считывает пробельный символ, ограничивающий заданное число слева. Там же она должна находиться в конце работы.

Программа, решающая поставленную задачу, имеет следующий вид:

$$R(R)L(r'(L(L))r'L)Rr.$$



- Вычитание единицы здесь вполне традиционное, знакомое по алгоритму вычитания столбиком.

Эзотерические языки программирования

- Языки программирования, разработанные для исследования границ возможностей их разработки, для доказательства потенциально возможной реализации некой идеи (так называемое «*доказательство концепции*», *proof of concept*), в качестве произведения программного искусства или в качестве шутки (компьютерного юмора).
- Многие эзотерические языки придумываются для развлечения, часто они пародируют «настоящие» или являются абсурдным воплощением «серьёзных» концепций программирования. Некоторые эзотерические языки нарочно ограничены, другие являются тьюринг-полными, то есть языками общего назначения.
- Общее свойство, присущее любому эзотерическому языку — текст программы на нём понятен лишь «посвящённому» либо непонятен вообще, потому что для составления программы нужно написать программу на обычном языке программирования. В то время как разработчики «реальных» языков программирования стараются сделать синтаксис максимально понятным, а программирование — удобным, создатели эзотерических языков ставят перед собой другие задачи.
- На практике такие языки, как правило, бесполезны, однако программирование на некоторых из них является неплохой тренировкой. Эзотерические языки нередко включают в список разрешённых языков на конкурсах по программированию.
- Большой класс составляют Brainf*ck-подобные языки, ориентированные на сокращение синтаксиса (оригинальный Brainf*ck имеет 8 команд) при сохранении тьюринг-полноты), среди таковых CaneCode, Ook! (язык орангутанов), COW (язык парнокопытных), Brainfork (многозадачный Brainf*ck), Whitespace, Spoon, LOLCODE, HQ9+ (не полон по Тьюрингу)…
- Среди языков, созданных для других целей, иногда упоминаемых как эзотерические —QuakeC, MC++, Автокод «Инженер» (низкоуровневый мнемокод для компьютера «Минск»)...

Модель $\mathcal{P}''$ и ее реализация

- Модель $\mathcal{P}''$ имеет реальное воплощение в виде настоящего языка программирования, который называется *Brainfu*k («вынос мозга»)*. Его команды однозначно отображаются на введённые здесь команды r, r', L и R, и обозначаются, соответственно, +, -, < и >. Круглые скобки заменяются на квадратные.
- Программа вычитания единицы будет выглядеть в этих обозначениях следующим образом:

  >[>]<[-[<[<]]-<]>+.
- Помимо этих команд в языке *Brainfu*k* имеются также команды ***. (вывод символа)*** и ***, (ввод символа)***.
- Таким образом, это полноценный (хоть и минималистичный) язык с вводом-выводом, на котором можно запрограммировать любую вычислимую функцию (разумеется, при условии наличия сколь угодно большого объёма памяти).

Brainfuck	P''
>	R
<	L=r'λ
+	r=λR
–	r'=rrrrr...rr (n раз)
[	(
]	)

- Придуман в 1993 году Урбаном Мюллером.

Что это???!!!

```
++++++++++++++++++++++++++++++++++++++++++++++++++
++++++++++++++++++++++++++.++++++++++++++++++
++++++++++++.+++++++..+++.----------------- ----------------
---------------------- -------------
.++++++++++++++++++++++++++++++
++++++++++++++++++++++++++.++++++++++++++++++
++++++.+++.------.-------.---------------- ------------------------
------------- ----.--------------------.
```

Ответ

- Пошаговая программа на языке Brainfu*k, печатающая «*Hello World!*» с переносом строки (в виде ASCII-кода - 72 101 108 108 111 32 87 111 114 108 100 33 10).
- Итого 389 операторов и использована 1 ячейка памяти.

- Оптимизированная программа заметно короче - всего 111 операторов, но 5 ячеек памяти. Первая ячейка используется как обратный счётчик цикла на 10 итераций:

```
++++++++++[>+++++++>++++++++++>+++>+<<<<-]>++
.>+.+++++++..+++.>++.<<+++++++++++++++.>.+++. ------
.--------.>+.>.
```

СПАСИБО
ЗА ВНИМАНИЕ